

DBMS PROJECT · VIVA PREPARATION

# Campus Wallet

Database Design, ER Model & Design Rationale

---

A student fintech platform on **PostgreSQL** · raw SQL (no ORM)

Immutable double-entry ledger · generalization/specialization · enforced-in-the-database integrity

**Contents:** System overview · Relational schema · ER model · Design decisions & rationale · Normalization analysis · Why the design is strong · Viva Q&A

# 1 · What the system is

---

**Campus Wallet** is a campus-payments platform. A student holds a spending wallet and a locked savings wallet, tops up from an external rail, pays campus bills (exam / hall / cafeteria), pays their share of **batch collection drives** (trips, events), is assessed **dues**, earns **loyalty points**, and saves through purchase **round-ups**.

The design goal is **financial integrity enforced by the database itself**, not by the application. Money can never be created, lost, or double-spent — the schema, constraints, triggers and functions guarantee it.

**Scale:** 31 base tables in total. The data model centres on **22 core tables** across six subsystems; nine supporting tables (a currency reference, role grants/requests, KYC records, notifications, the audit log, a state-machine reference, loyalty config, and one link table) are omitted from the diagrams so the *core data model* reads clearly.

**One-line architecture.** Every unit of money moves through an **immutable, append-only, double-entry ledger**; balances are **derived** (not stored as truth); and every payable — student wallet, campus till, event pool — is a **specialization of one common account** .

## 2 · The relational schema

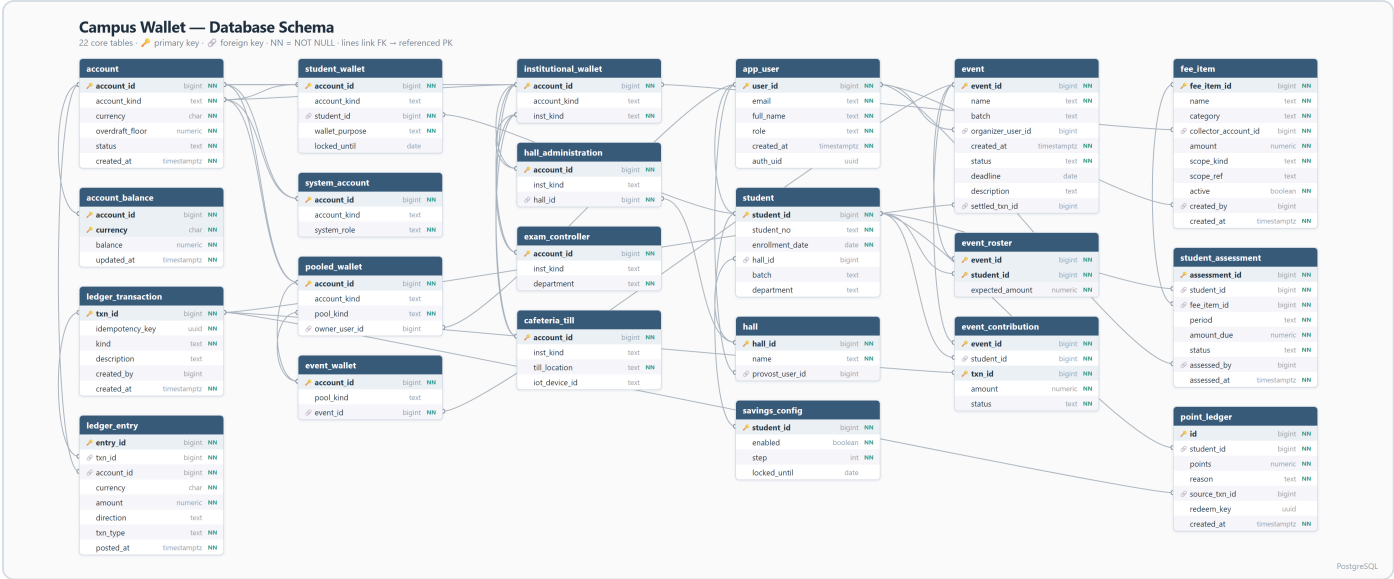


Figure 1 — Relational schema (22 core tables). Navy = table header; 🔑 primary key; 🔗 foreign key; NN = NOT NULL; lines link each foreign key to the primary key it references.

The tables group into six subsystems:

| Subsystem                              | Tables                                                                                                                                      | Purpose                                                                                             |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Core Ledger                            | account, account_balance, ledger_transaction, ledger_entry                                                                                  | The money spine: signed, balanced legs posted to accounts; a derived balance cache.                 |
| Wallets<br>(specialization of account) | student_wallet, system_account, pooled_wallet → event_wallet, institutional_wallet → hall_administration / exam_controller / cafeteria_till | Every kind of "money holder" is a subtype of <b>account</b> — a two-level generalization hierarchy. |
| Users & Identity                       | app_user, student, hall, savings_config                                                                                                     | People and their academic identity; per-student round-up settings.                                  |
| Events & Collection Drives             | event, event_roster, event_contribution                                                                                                     | Batch collection drives with a roster and a live defaulter list.                                    |
| Fees & Dues                            | fee_item, student_assessment                                                                                                                | A fee catalogue and the per-student assessed dues.                                                  |
| Loyalty                                | point_ledger                                                                                                                                | An append-only points ledger (earn on spend, redeem to cash).                                       |

### 3 · The entity–relationship model

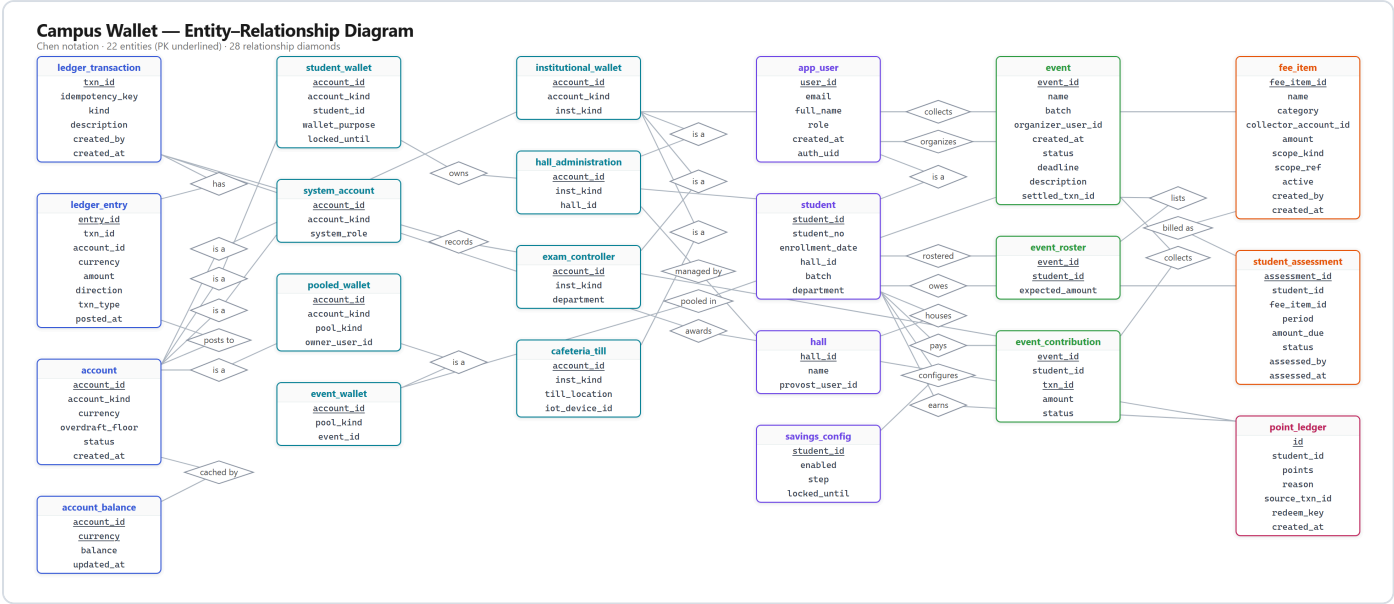


Figure 2 — ER diagram (Chen notation). Rectangles = entities (primary key underlined); diamonds = relationships; "IS-A" diamonds show generalization/specialization.

The hubs of the model are `account` (every wallet *is-a* account), `app_user` / `student` (every actor), and `ledger_transaction` (every money movement). Relationships carry their real cardinality — e.g. one `ledger_transaction` *has* many `ledger_entry` legs (minimum two); one `event` *lists* many roster rows; a `student` *owes* many `student_assessment` dues.

Junction/associative entities ( `event_roster` , `event_contribution` , `student_assessment` ) resolve many-to-many relationships and carry their own attributes (expected amount, paid amount, due amount).

## 4 · Design decisions & why we took them

### D1 — A double-entry, append-only ledger (not a balance column)

**Why:** A single `balance` column can drift, be edited, or lose the audit history of how money moved. Double entry makes every movement a set of signed legs that must sum to zero, so value is conserved by construction.

**How it's enforced:** `ledger_transaction` (the business event) has  $\geq 2$  `ledger_entry` legs with signed `amount` (+credit / -debit). `UPDATE` / `DELETE` / `TRUNCATE` on ledger tables raise an exception (append-only); corrections are compensating reversal transactions.

### D2 — Balances are DERIVED, with an O(1) cache and nightly reconciliation

**Why:** The ledger is the single source of truth, but summing every leg on each read is slow. We keep a cache for O(1) reads without making it authoritative.

**How it's enforced:** `account_balance` is maintained by an `AFTER INSERT` trigger on `ledger_entry`. It is a **declared, deliberate denormalization**. A reconciliation query proves `cache = SUM(ledger)` for every account using set difference (`EXCEPT`); any drift row is a bug.

### D3 — Generalization / Specialization: one account, many wallet subtypes (disjoint & total)

**Why:** Student wallets, system accounts (treasury, loyalty pool, external rail), event pools and institutional tills all behave like accounts (they hold a balance, post legs) but have distinct attributes. Modelling them as subtypes avoids one wide, mostly-NULL table and lets new payee types slot in.

**How it's enforced:** `account` carries a discriminator `account_kind` and a composite `UNIQUE(account_id, account_kind)`. Each subtype pins `account_kind` as a `GENERATED` constant and foreign-keys the composite — so an account **physically cannot appear in two subtypes** (declarative disjointness, zero triggers). **Totality** (every account has exactly one subtype) is checked at COMMIT by a `DEFERRABLE` constraint trigger. Two levels: `account`  $\rightarrow$  `institutional_wallet`  $\rightarrow$  {hall, exam, cafeteria}.

### D4 — Exactly-once money movement (idempotency keys)

**Why:** A retried or double-submitted request (two tabs, a network retry) must never move money twice.

**How it's enforced:** `ledger_transaction.idempotency_key` is `UNIQUE`. The sole money-mover `make_transfer()` does `INSERT ... ON CONFLICT DO NOTHING` and returns the original transaction on replay. Deposits/refunds additionally verify the replayed txn actually credits the right account before trusting the key.

### D5 — Zero-sum & minimum-two-legs enforced at COMMIT (deferred constraint)

**Why:** Multi-leg transactions are built with several INSERTs; the balanced invariant can only be true once all legs exist.

**How it's enforced:** A `DEFERRABLE INITIALLY DEFERRED` constraint trigger checks, per currency, that `SUM(amount)=0` for each transaction at COMMIT. A single non-zero leg can't net to zero, so this also enforces the  $\geq 2$ -leg rule. A companion deferred check forbids a legs-less transaction header.

## D6 — The live defaulter list is a SET OPERATION (relational algebra in action)

**Why:** Who still owes money must be correct under partial payments and refunds, in real time, without a status column that can go stale.

**How it's enforced:** `v_event_defaulters = roster EXCEPT fully-paid`. A partial payer never enters *fully-paid*; a refund posts a **negative** contribution that drops their sum back below expected, so they automatically re-appear. Collection progress uses a `RANK() OVER()` window function.

## D7 — Snapshotting: dues store the amount at assess-time (controlled, justified redundancy)

**Why:** If a fee's price later changes, an already-assessed due must keep the price the student actually owes — it is a historical fact, not a live lookup.

**How it's enforced:** `student_assessment.amount_due` and `event_roster.expected_amount` are snapshots. This is **not** a normalization violation: the value is not functionally derivable from the current catalogue — it records a point-in-time obligation.

## D8 — Security in the database: least-privilege roles, a SECURITY-DEFINER funnel, and RLS

**Why:** The application role must not be able to hand-edit the ledger or read other students' wallets, even if application code is buggy or bypassed.

**How it's enforced:** Three roles ( `app_read` / `app_write` / `app_admin` ) hold only `SELECT`; every mutation funnels through `SECURITY DEFINER` functions owned by a privileged role. Row-Level Security isolates student-owned rows so a session only sees its own wallets.

## D9 — A generic audit trail

**Why:** Sensitive tables (fees, assessments, roles) need a tamper-evident change history for accountability.

**How it's enforced:** One reusable `audit()` trigger writes `to_jsonb(OLD)` / `to_jsonb(NEW)`, the action, actor and timestamp into `audit_log` — the ledger itself is immutable, so it is *not* audited (that would duplicate history).

## D10 — Consolidation: we dropped the Clubs feature and scoped the diagrams

**Why:** For the presentation the model must fit a slide and show the *core* design, not every supporting table. The Clubs feature (`club`, `club_member`, an INTERSECT demo) was unused, so removing it is a genuine simplification — not a denormalization.

**How it's enforced:** A migration drops `club` / `club_member` and makes drives batch-only; the diagrams additionally hide nine reference/infrastructure tables (currency, role grants, KYC, notifications, audit, state-machine, loyalty config, link table). We deliberately **kept** the specialization hierarchy and the ledger — those are the graded DBMS concepts, and denormalizing them to cut boxes would weaken the design.

## 5 · Normalization analysis

---

The operational tables are in **BCNF**: every non-trivial functional dependency has a candidate key on its left-hand side.

- `ledger_entry(entry_id → txn_id, account_id, currency, amount, ...)` — the only determinant is the key; no partial or transitive dependencies.
- `student_assessment(assessment_id → student_id, fee_item_id, period, amount_due, status)` with an alternate key `(student_id, fee_item_id, period)`; both are candidate keys → BCNF.
- `event_roster(event_id, student_id → expected_amount)` — a pure associative entity; the whole key determines the one attribute.
- Reference data (currency, halls, fee catalogue) is factored out to avoid repeating names/rates across rows.

### The two deliberate exceptions — and why they're correct

- **account\_balance (a materialized cache)**. Technically derivable from `SUM(ledger_entry.amount)`, so storing it is redundancy. It is a *declared* performance denormalization, trigger-maintained and reconciled against the ledger — the standard, defensible trade-off.
- **Snapshot amounts** (`amount_due`, `expected_amount`). These look like duplication of the catalogue price but are *historical obligations*, not derivable from current data — so they do not violate normalization.

## 6 · Why this design is strong

---

- **Integrity by construction.** Value is conserved (per-currency zero-sum), the ledger is append-only, and per-account overdraft floors stop unauthorised negatives — enforced by constraints/triggers, not by hopeful application code.
- **Correct under concurrency.** Idempotency keys give exactly-once semantics; transfers lock accounts in a deterministic order (LEAST/GREATEST id) to avoid deadlocks; sensitive flows run `SERIALIZABLE`.
- **Performance where it matters.**  $O(1)$  balance reads via the cache; partial and targeted indexes (e.g. open dues, ledger-by-account-time).
- **Security in depth.** Least-privilege roles + a SECURITY-DEFINER write funnel + Row-Level Security.
- **Extensible.** The account/wallet specialization means a new payee or wallet type is a new subtype, not a schema rewrite.
- **Demonstrates advanced SQL.** Set operations ( `EXCEPT` ), window functions ( `RANK()` ), CTEs, generated columns, deferred constraint triggers, composite/disjoint foreign keys, and views with `security_invoker`.

## 7 · Anticipated viva questions & answers

---

### Q. Why double-entry instead of a simple balance column?

A balance column is a single mutable number with no history — it can drift or be edited and you can't prove how it got there. Double entry records every movement as balanced signed legs, so value is conserved by construction and the full history is reconstructable. The balance becomes a *derived* quantity you can always re-compute.

### Q. If balances are derived, why keep `account_balance` ?

Purely for O(1) reads. It's a declared cache, maintained by a trigger and **reconciled** against `SUM(ledger)` — the ledger stays the single source of truth. It's a deliberate, justified denormalization, not an accidental one.

### Q. How do you guarantee an account can't be two wallet kinds at once (disjointness)?

Declaratively. `account` has `UNIQUE(account_id, account_kind)` ; each subtype fixes `account_kind` as a generated constant and foreign-keys that composite. So a row physically can't satisfy two subtypes — no trigger required. Totality (every account has exactly one subtype) is a deferred constraint trigger checked at COMMIT.

### Q. How is exactly-once payment guaranteed?

Every transaction carries a `UNIQUE` idempotency key. The money-mover inserts `ON CONFLICT DO NOTHING` and, on a replay, returns the original transaction instead of posting again — so a retry or double-submit is a no-op.

### Q. Why is the zero-sum check deferred to COMMIT?

A multi-leg transaction is inserted one leg at a time; it is only balanced once all legs exist. A `DEFERRABLE INITIALLY DEFERRED` constraint trigger lets the legs be inserted mid-transaction and verifies `SUM=0` per currency at COMMIT.

### Q. How does the defaulter list stay correct after a refund?

It's `roster EXCEPT fully-paid` . A refund posts a negative contribution, dropping the student's paid-sum below their expected amount, so they drop out of *fully-paid* and re-appear in the EXCEPT result automatically — no status flag to update.

### Q. Isn't storing `amount_due` redundant with the fee's price?

No — it's a snapshot of what the student owed *at assess time*. If the catalogue price changes later, the existing due must not change. It's a historical fact, not a value derivable from current data, so it isn't a normalization violation.

### Q. What normal form is the schema in, and are there exceptions?

The operational tables are in BCNF. The only intentional exceptions are the `account_balance` cache (a reconciled performance denormalization) and the snapshot amounts (historical, not derivable).

### Q. How do you stop one student reading another's wallet?

Row-Level Security. The app roles hold only SELECT and see only rows owned by the current session user; all writes go through SECURITY DEFINER functions, so the app can never hand-edit the ledger.

### Q. Why funnel every write through functions instead of granting INSERT?

So money can only move one sanctioned way. The app role has no direct DML on the ledger; the SECURITY DEFINER functions are the single audited, validated entry point — this is defence-in-depth and makes the invariants unbyassable.

**Q. How are deadlocks avoided on concurrent transfers?**

Both accounts are locked in a deterministic order — `LEAST(from,to)` then `GREATEST(from,to)` — so two concurrent transfers over the same pair can't acquire locks in opposite orders.

**Q. Where are the set operations and window functions?**

EXCEPT builds the live defaulter list and the balance reconciliation; RANK() OVER() ranks collection progress; INTERSECT was used for cross-club overlap before that feature was removed.

**Q. Why did you drop the Clubs tables?**

The feature was unused and drives are batch-scoped, so removing `club / club_member` is a real simplification. Crucially we kept the specialization hierarchy and ledger rather than collapsing them — those demonstrate the DBMS concepts and denormalizing them just to shrink the diagram would weaken the design.

**Q. If you had to add a new payee type (say a library), what changes?**

Just a new subtype under `institutional_wallet` — a small table pinning its `inst_kind` and its own attributes. The ledger, balances, payment flow and payable-targets view need no change. That extensibility is the whole point of the specialization design.